List of Experiments

Total No of Labs: 14

S. No.	Programs List	No. of Labs
CAD Experiments		
1	Study of basic graphics functions defined in “graphics.h”.	1
2	To implement Digital differential analyzer (DDA) Algorithm for drawing a line segment between two given end points A (x1, y1) and B(x2, y2).	1
3	To implement Bresenham’s line drawing algorithm for drawing a line segment between two given endpoints A (x1, y2) and B(x2, y2).	1
4	To implement Bresenham’s Cicrcle drawing algorithm for drawing a circle with a given center of circle P(x, y) and radius r.	1
5	To apply the basic 2D transformations such as translation, Scaling, Rotation, shearing and reflection for a given 2D object.	2
6	To apply the basic transformations such as translation, Scaling, Rotation for a given 3D object.	2
7	To generate a smooth curve by using Bezier curve technique for a given set of control points.	1
8	To implement Cohen–Sutherland 2D clipping and window–viewport mapping	1
Product Design		
9	Introduction to CAD software’s.	1
10	Generation of any one simple solid model using any CAD software.	1
11	Generation of any one assembly of mechanical part using any CAD software.	1
12	Design and develop the complete lifecycle for any product.	1

Experiment - 2

Aim: To implement Digital differential analyzer (DDA) Algorithm for drawing a line segment between two given end points A (x1, y1) and B(x2, y2).

Description: DDA algorithm is an incremental scan conversion method. Here we perform calculations at each step using the results from the preceding step. The characteristic of the DDA algorithm is to take unit steps along one coordinate and compute the corresponding values along the other coordinate. The unit steps are always along the coordinate of greatest change, e.g. if dx = 10 and dy = 5, then we would take unit steps along x and compute the steps along y.

In DDA we need to consider two cases;

One is slope of the line less than or equal to one ($|m| \leq 1$) and slope of the line greater than one ($|m| > 1$).

1) When $|\mathbf{m}| \leq \mathbf{1}$ means $y_2-y_1 = x_2-x_1$ or $y_2-y_1 < x_2-x_1$.In both these cases we assume x to be the major axis. Therefore we sample x axis at unit intervals and find the y values corresponding to each x value. We have the slope equation as

   $\Delta y = m \Delta x$

   $y_2-y_1 = m (x_2-x_1)$

   In general terms we can say that $y_{i+1} - yi = m(x_{i+1} - x_i)$. But here $\Delta x = 1$; therefore the equation reduces to $y_{i+1} = y_i + m = y_i + dy/dx$.

2) When $|\mathbf{m}| > \mathbf{1}$ means $y_2-y_1 > x_2-x_1$ and therefore we assume y to be the major axis. Here we sample y axis at unit intervals and find the x values corresponding to each y value. We have the slope equation as

   $\Delta y = m \Delta x$

   $y_2-y_1 = m (x_2-x_1)$

Algorithm

1. Start.

2. Declare variables x,y,x1,y1,x2,y2,k,dx,dy,s,xi,yi and also declare gdriver=DETECT,gmode.
3. Initialise the graphic mode with the path location in TC folder.
4. Input the two line end-points and store the left end-points in (x1,y1).
5. Load (x1,y1) into the frame buffer;that is,plot the first point.put x=x1,y=y1.
6. Calculate dx=x2-x1 and dy=y2-y1.
7. If abs(dx) > abs(dy), do s=abs(dx).
    a. Otherwise s= abs(dy).
8. Then xi=dx/s and yi=dy/s.
9. Start from k=0 and continuing till k<s,the points will be
    a. x=x+xi.
    b. y=y+yi.
10. Place pixels using putpixel at points (x,y) in specified colour.
11. Close Graph.
12. Stop.

Programme:

```
#include<stdio.h>
#include<graphics.h>
#include<math.h>
float round(float a);
void main()
{
int gd=DETECT,gm; // gd=graphics driver (detects best graphics driver and assigns it as default,
gm=graphics mode.)
int x1,y1,x2,y2,steps,k;
float xincr,yincr,x,y,dx,dy;
printf("enter x1,y1");
scanf("%d%d",&x1,&y1);
printf("enter x2,y2");
scanf("%d%d",&x2,&y2);
initgraph(&gd,&gm,"c:\\turboc3\\BGI");//initializes the graph
dx=x2-x1;
dy=y2-y1;
if(abs(dx)>abs(dy))
```

```
steps=abs(dx);
else
steps=abs(dy);
xincr=dx/steps;
yincr=dy/steps;
x=x1;
y=y1;
for(k=1;k<=steps;k++)
{
delay(100);//for seeing the line drawing process slowly.
x+=xincr;
y+=yincr;
putpixel(round(x),round(y),WHITE);
}
outtextxy(200,20,"DDA"); // for printing text at desired screen location.
outtextxy(x1+5,y1-5,"(x1,y1)");
outtextxy(x2+5,y2+5,"(x2,y2)");
getch();
closegraph(); // closes the graph and comes back to previous graphic mode.
}
float round(float a)
{
int b=a+0.5;
return b;
}
```